PACKAGE 07 / VOLUME 06

Module 30: Production quality and the undergraduate engine capstone

Four expanded lessons • Project and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Module 30: Production quality and the undergraduate engine capstone 4

Lesson 30.1: Profiling, frame budgets and performance regression tests . . . 5

Lesson 30.2: Compatibility, accessibility, localization and human testing . . . 8

Lesson 30.3: Packaging, patching, modding and supply-chain boundaries . . . 11

Lesson 30.4: Capstone review, release and technical postmortem . . . 14

Module 30 project . . . 16

Module 30 review . . . 17

Reference Directory 18

S01: Collision documentation . . . 18

S02: Simulation documentation . . . 18

S03: glTF tutorial: Skins . . . 18

S04: glTF 2.0 specification . . . 18

S05: Writing a callback function . . . 18

S06: SDL_AudioStreamCallback . . . 18

S07: Xbox Accessibility Guidelines . . . 18

S08: CS188: Informed search . . . 19

S09: Introduction . . . 19

S10: CS188: Minimax . . . 19

S11: Automatic differentiation with torch.autograd . . . 19

S12: Networked movement in the Character Movement Component . . . 19

S13: Snapshot Interpolation . . . 19

S14: GGPO SDK repository 19

S15: Transaction documentation 20

S16: Atomic Commit in SQLite 20

S17: os.replace documentation 20

S18: zipfile documentation 20

S19: Fix Your Timestep! 20

S23: XAG 103: Additional channels for audio and visual cues 20

S24: Deterministic Lockstep 20

S25: XAG 107: Input 20

S26: XAG 101: Text display 21

Original program reading IDs 21

Module 30: Production quality and the undergraduate engine capstone

Prerequisites: Modules 21-29 plus Gate D software-engineering evidence.

Original reading IDs: R20, R26, R29, R45. See the reference directory.

Module goal: Gate F: release a small original game using your engine, with one complete gameplay loop, settings, save/load, diagnostics, accessibility checks, a tested distribution and a technical postmortem. Scope is judged by quality of evidence, not visual size.

Lesson 30.1: Profiling, frame budgets and performance regression tests

Learning goal: Optimize measured bottlenecks without sacrificing correctness.

Profile the complete experience

A game can average 60 frames per second and still stutter. Store raw frame durations, not only an average FPS counter. Describe what each timer measures: CPU simulation, render submission, GPU execution, present wait or end-to-end frame interval. CPU and GPU work may overlap, so summing every timer can double-count elapsed time. Preserve the timing boundaries established in Volume 5.

Choose a fixed benchmark scene, input tape, resolution, build configuration and warm-up policy before optimizing. Record CPU/GPU model where applicable, operating system, compiler, flags, power conditions and background load. A software-renderer path is valid when clearly labeled. Do not claim GPU timing from a CPU timer around an asynchronous submission.

Distributions and measurement boundaries

A nearest-rank percentile sorts N observations and selects index ceil(p*N)-1 for p in (0,1]. Different statistical packages may use interpolation instead, so record the convention. For durations [10,10,10,10,50] ms, nearest-rank p95 is 50 ms and the mean is 18 ms. A tiny sample is useful for checking code but weak evidence about a production tail distribution.

With 1,000 samples, nearest-rank p99 uses rank 990. Keep the raw series so a reviewer can distinguish isolated stalls, periodic spikes and warm-up effects. Report multiple independent runs and the number of samples, not a single impressive minimum. Timing noise is not fixed by printing more decimal places.

Amdahl's law as a scope check

If a 20 ms frame spends 2 ms in a subsystem, making that subsystem ten times faster reduces it to 0.2 ms. The unchanged 18 ms remains, so the ideal new frame is 18.2 ms, a speedup of about 1.099. Even an infinitely fast replacement cannot reduce this modeled frame below 18 ms. This arithmetic assumes the measured work is on the relevant critical path and that optimization does not change other costs.

A microbenchmark can improve while the frame gets slower because of extra allocations, cache pressure, synchronization or data conversion elsewhere. Measure before and after in the same workload and verify correctness. A faster collision routine that drops contacts is not an optimization. A faster model that changes decision quality needs a quality/performance comparison, not a pure speed claim.

Regression thresholds need context

A useful performance test has a declared workload and an acceptance rule. For a stable dedicated machine, a percentile threshold may be meaningful. On shared CI, use broader thresholds, repeated runs and a review path for noisy failures. Keep functional tests separate so performance noise does not obscure correctness regressions. A budget threshold is a project target, not a universal statement about what every machine should achieve.

Track work counts alongside time: bodies, contacts, active voices, navigation expansions, rollback ticks, draw calls and bytes loaded. A frame spike after adding 100 enemies may be expected scaling rather than a compiler regression. Normalize only when the normalization answers the question; hiding a doubled workload behind 'time per entity' can conceal a worse player experience.

Worked optimization experiment

Suppose a deterministic scene spends most simulation time rebuilding an unchanged navigation grid. Hypothesis: versioning the static grid will remove repeated work. First retain the original implementation as a correctness oracle. Run the same input tape through both and compare paths and gameplay state. Then collect several before/after frame series with identical warm-up and configuration. Record any added memory for cached data and how invalidation works when a door changes.

If the result improves median time but increases rare spikes due to cache rebuilds, report both. Consider whether rebuilding incrementally or at a controlled loading boundary better fits the game. The best result is a measured design tradeoff, not the largest isolated speedup percentage.

Deliberate practice

The kernel calculates finite, positive frame-time statistics using the declared nearest-rank rule. The full lab profiles the capstone, optimizes one verified bottleneck and stores before/after raw data. Include a correctness comparison, repeated runs, peak-memory measurement and a limits statement. Synthetic demo timings are labeled synthetic; locally measured test-harness timings are not presented as capstone performance.

Build a regression test that another person can run from the package. Explain one change that failed to improve performance and why the evidence changed your plan. A negative optimization result can demonstrate stronger engineering judgment than an unsupported claim of a dramatic speedup.

Original worked checkpoint

A subsystem taking 2 ms of a 20 ms frame can save at most 2 ms if removed entirely. A 10x speedup in that subsystem alone yields an 18.2 ms frame under the simplified additive model.

Full practical assignment

Profile the capstone scene, optimize one verified bottleneck and add a regression benchmark. Pass: repeated before/after results, unchanged correctness and disclosed measurement uncertainty are included.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Nearest-rank frame statistics**. Run `starter/python/demos/lesson_30_1.py`; implement `starter/python/exercises/lesson_30_1.py`.

Assigned study and native transfer

R26 : profiling and optimization; R45 : sanitizer diagnostics; revisit Modules 09 and 18.

Revisit R20 production and R26 profiling concepts; read S19 for the distinction between simulation catch-up and rendering cadence. Preserve Volume 5 CPU/GPU timing definitions.

Engine transfer: Add a reproducible capstone benchmark with raw frame durations, work counts and environment metadata. Keep measurement code out of the authoritative replay hash.

Check your understanding

1. What is the ideal frame time after a 10x speedup of the example 2 ms subsystem?
2. Why record build type?
3. Why can a microbenchmark improvement fail to help the game?
4. Using nearest-rank percentiles, what are the mean and p95 of [10,10,10,10,50] ms? Is this enough data to characterize a real game tail?
5. A 20 ms frame contains 2 ms of optimizable work. What is the ideal total time after a 10x improvement, and what is the infinite-speed lower bound?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.2: Compatibility, accessibility, localization and human testing

Learning goal: Test whether people can actually use the complete product.

Test supported experiences, not a vague platform list

A compatibility matrix states exactly what was tested: operating system and version, display size, input device, audio availability, renderer/backend and build configuration. 'Works on PC' is not a test result. Mark untested combinations as untested rather than inferred. A minimum supported configuration is a project decision backed by measurements and task tests.

Prioritize the full gameplay loop: start, understand controls, perform the objective, recover from failure, save where supported, load and exit. Test a clean folder without development tools or hidden asset paths. Missing controller or audio devices should produce a usable fallback rather than a crash. Resizing, losing focus and exiting while loading are state transitions with ownership implications.

Input is an action map

Bind physical controls to semantic actions such as move_left or interact. The UI asks the binding system for the current display label; it does not hard-code 'Press E.' Rebinding must update gameplay, prompts, menus and saved settings consistently. Define conflict handling and a way to restore defaults. A user must not be able to bind away every route to the settings screen without a recovery path.

Avoid requiring simultaneous or held inputs when an alternate mode can preserve the intended experience. Test keyboard-only menu navigation and focus visibility. Microsoft XAG 107 provides the assigned input-design reference [S25]. A checklist does not establish that every player can use the game; observe the chosen tasks with appropriate participants.

Localization begins before translation

Store text as keyed resources with placeholders, not as scattered string literals. Format numbers, dates and plurals through an appropriate locale-aware layer where needed. Pseudo-localization expands or alters strings to expose clipping, concatenation assumptions and hard-coded text. Preserve placeholders such as {player} while expanding surrounding words.

A simple teaching pseudo-localizer can bracket text and add expansion characters. It is not a translator, font-shaping engine or full internationalization library. Test long labels, multiline text, missing keys and right-to-left layout when included in the support scope. Unicode code points, grapheme clusters and display width are not interchangeable. The full lab should use the engine's real layout system rather than predicting fit from character count alone.

Worked binding and text test

The default interaction key is E. Rebind it to Space and reopen the objective screen. The prompt must now identify Space, including after save/reload. Expand 'Activate relay' to a longer test string and reduce the window size. The control must remain readable and operable, perhaps through wrapping or layout growth. Shrinking text until it technically fits can violate the intended readability target.

Mute audio and repeat the relay task without coaching. A visual status change and readable message should carry the critical cue. Remove color as the sole distinction between inactive and active states. Microsoft XAG 101 addresses text display, and XAG 103 addresses additional information channels [S26, S23]. Record specific observations rather than claiming broad accessibility certification.

Human testing as evidence

Use voluntary participation, explain the task and collect only needed observations. A small informal test is useful for finding defects but is not a representative population study. Ask the tester to complete a task without step-by-step coaching, then record where they hesitated, failed or misunderstood. Do not treat polite enjoyment feedback as proof that controls are discoverable.

Write each issue with reproduction steps, affected configuration, severity, evidence and a retest result. 'Player got confused' is too vague; 'after rebinding interact, the objective prompt still showed E, causing three failed attempts' points to a fix. Do not store unnecessary identifying information in the report.

Deliberate practice

The kernel pseudo-localizes literal text while preserving brace placeholders and validates a small action-binding map. The full lab runs at two window sizes, with remapped controls, muted audio, missing devices and expanded text. Include at least one independent task test and retest a discovered issue. Mark user-testing evidence as pending until a real tester participates.

For the capstone, integrate settings early rather than adding them after every UI string and input check has been hard-coded. Readability, recovery and control customization are part of a finished gameplay loop. They are not cosmetic polish that can be waived because the physics tests pass.

Original worked checkpoint

A button sized for a short English label can clip a longer translated label. A control prompt that hard-codes a keyboard letter can become wrong after rebinding.

Full practical assignment

Test the game at two window sizes, with remapped input and an expanded pseudo-localized text set. Pass: no critical clipped controls, a muted-audio task remains solvable and observed usability issues are tracked.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Pseudo-localization and current binding prompt**. Run `starter/python/demos/lesson_30_2.py`; implement `starter/python/exercises/lesson_30_2.py`.

Assigned study and native transfer

R29 : input, text, audio, contrast and UI guidelines; R20 : playtesting and production.

Read S25 input, S26 text display and S23 additional channels. Read S07 for the limits of guideline-based review. Use the original R20 playtesting material for task observation.

Engine transfer: Centralize action bindings and text resources in the engine. Have an independent tester complete the loop with a changed binding and muted audio; record and retest actual issues.

Check your understanding

1. Why use pseudo-localization before full translation?
2. What should a control prompt show after rebinding?
3. Does checking every accessibility guideline prove legal compliance?
4. After interact is rebound from E to Space, which systems must reflect the change?
5. Why is an expanded pseudo-localized label useful but insufficient to prove support for all languages?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.3: Packaging, patching, modding and supply-chain boundaries

Learning goal: Distribute a game without creating an uncontrolled execution path.

A release is a reproducible boundary

The release folder must contain the executable or supported runtime entry point, required assets, defaults, notices, dependency information and launch instructions. It must not depend on an absolute path in the developer's home directory. Keep user saves outside replaceable installation files. Test an extracted package in a new folder before declaring it runnable.

An archive checksum detects a change in bytes when compared to a trusted expected value. A checksum shipped alongside an untrusted archive does not by itself authenticate the publisher; an attacker could replace both. Distinguish integrity, authenticity and authorization. The capstone requires provenance and integrity records, not a custom cryptographic update system invented without review.

Validate data-only mods before allocation

A data-only mod can still consume excessive memory, trigger integer overflow in native code or exploit a parser defect. Define a small versioned schema with allowlisted fields and bounded lengths, counts and dimensions. Reject unknown executable hooks. Check multiplication before allocating width*height*channels bytes. In a fixed-width language, use checked arithmetic or compare against the maximum before multiplication.

The core schema permits a title and a bounded list of resource dimensions; it does not execute scripts or extract arbitrary files. For archives, reject absolute paths, parent traversal, device paths and unexpected links, and enforce uncompressed-size and file-count limits before extraction. Python's ZIP documentation explicitly warns about untrusted archives and extraction concerns [S18]. The included validator is a narrow teaching contract, not a general secure archive extractor.

Worked allocation check

A texture declaring width=100000, height=100000 and four bytes per pixel requests 40,000,000,000 bytes, approximately 40 GB in decimal units. Even without integer overflow, this exceeds a small game's resource budget. Check each dimension and the total resource budget before allocation. Multiple individually acceptable textures can still exceed a package-wide cap, so validate the sum as well.

A mod schema version of 99 is not silently treated as version 1. An unsupported version should fail with a useful error before altering the current game state. Parse into a temporary validated object, then install it at a safe boundary. A partially applied mod is harder to recover from than a rejected one.

Interrupted updates and saves

Use staged version directories rather than overwriting the only valid installation in place. Build a candidate version, verify its manifest, test its entry point where feasible, then switch a small active-version pointer. Retain the prior version until the new one is accepted. If staging fails, the active version stays unchanged. This teaching model does not claim a production updater across all filesystems.

For a save file, write a temporary file in the same directory, flush it as appropriate, then replace the destination. Atomic replacement provides a useful visibility property on supported same-filesystem operations; durability across power loss also depends on flushing, directory metadata, filesystem and operating-system behavior. Python's os.replace documentation describes the rename/replace contract, not a universal crash-durability guarantee [S17]. Keep a version and integrity check inside the save format and preserve a known-good copy when the design needs one.

Supply-chain and capability boundaries

Record library versions, source locations, licenses and whether a dependency is bundled or system-provided. Review build scripts before running them. Never automatically download and execute arbitrary community code as a convenience feature. Executable plugins run with the permissions of their process unless meaningful isolation says otherwise. Calling a component a sandbox is not evidence of a capability boundary.

The capstone's mod requirement is deliberately data-only. A script or native-plugin extension is advanced optional work requiring an explicit threat model, resource limits and a separate review. Do not expand a small finished game into an unbounded marketplace project.

Deliberate practice

The kernel validates a JSON-like mod object and calculates resource limits without allocating its declared payloads. The integration companion stages a candidate update and simulates interruption before activation. The full lab must package the actual game, test a clean-folder launch, reject malformed/oversized mods and preserve a valid version through the stated interruption tests. Include interrupted-save behavior and prove that replacing application files does not erase user progress.

Submit a release manifest, provenance list, schema, rejection corpus and recovery report. A green unit test for one validator does not prove arbitrary external content is safe; state exactly which inputs and capabilities are supported.

Original worked checkpoint

A malformed mod declares a texture with dimensions large enough to overflow a byte-count calculation. Validation must check multiplication and total resource limits before allocation, even though the file is nominally data.

Full practical assignment

Package the game and implement a data-only mod with a versioned schema. Pass: a clean-folder launch works, oversized or malformed mods are rejected and an interrupted-update simulation preserves a valid version.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Bounded data-only mod schema**. Run `starter/python/demos/lesson_30_3.py`; implement `starter/python/exercises/lesson_30_3.py`.

Assigned study and native transfer

R20 : production; R26 : packaging and plugins; R45 and Module 20: defensive memory and dependency practices.

Read S17 os.replace and S18 ZIP extraction warnings. Revisit Module 20 dependency and trust-boundary practices and original R45 defensive native-code material.

Engine transfer: Create a versioned data-only mod schema and an actual release folder. Test rejected input, interrupted staging and a save that remains intact when installation files are replaced.

Check your understanding

1. Can a data-only mod still be dangerous to robustness?
2. Why separate saves from replaceable installation files?
3. Does naming a system a sandbox establish isolation?
4. A texture declares 100000 by 100000 pixels at four bytes per pixel. What allocation would it imply, and when should it be rejected?
5. Why does a SHA-256 checksum stored beside a downloaded archive not by itself prove publisher authenticity?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.4: Capstone review, release and technical postmortem

Learning goal: Demonstrate ownership of a complete engineering result.

Finish one loop and own the result

Gate F is a small original game built on the learner's engine, not a collection of unrelated demonstrations. A loop has an entry state, a goal, meaningful player actions, success or failure, and a restart or continuation path. The scope can be a single arena, puzzle or short tactical scenario. Commercial art quality, revenue and a large world are not the assessment target.

Carry forward the Volume 5 engine ownership, rendering and resource evidence. Integrate collision/physics, animation or articulated motion, bounded audio, an inspectable AI, settings, save/load and a tested package. The multiplayer requirement may remain a separate local two-client demonstration with delay, loss and replay tests. Do not force a public online service into the game merely to satisfy the networking module.

Integration reveals contract mismatches

The physics system may use meters while the renderer uses pixels; animation may propose root motion while movement also applies velocity; audio may receive an event after an entity has been destroyed. A capstone architecture review should identify who owns each state, which thread or tick phase may mutate it, and how data crosses the boundary. An attractive diagram without a failure trace is not enough.

Freeze a small feature set before the final evidence run. Record deferred work explicitly. Adding a new renderer, account system or mod marketplace during stabilization undermines the goal. A well-tested small system is stronger evidence than a large unfinished system whose critical paths remain unexercised.

Evidence must be independently reproducible

Provide source, build commands, assets with provenance, a release package, raw performance data, test results and known limitations. A second person must be able to build or launch from a clean checkout or extracted source tree using documented prerequisites. Missing tools should produce a clear error rather than a silently skipped test that reports success.

The reviewer finishes the full gameplay loop, reproduces a named bug regression and observes a live change. A recording can supplement these checks but cannot replace independent execution. The learner must explain supplied reference code, library use and AI assistance. The submitted work should distinguish what was authored, adapted, generated and tested.

Worked release decision

Suppose the game completes its loop, but saving during scene loading can corrupt the only save. The average frame time meets the chosen budget and the art looks polished. This is still a release blocker because ordinary use can destroy progress. Fix the ownership/atomicity defect, or disable the unsafe save path with a clear supported behavior and retest. A high score in graphics cannot compensate for a critical data-loss path.

Now suppose one optional high-quality shadow mode misses the target budget on an untested GPU. Do not claim that configuration is supported. Restrict the support matrix to measured configurations and preserve a functional fallback. Honest scope restriction is different from concealing a defect in a claimed supported path.

Technical postmortem

Write the postmortem around decisions and evidence. State the initial hypothesis, what was implemented, what measurement or test challenged it, and what changed. Include at least one failed approach. For example, a spatial grid may have been slower than brute force in a tiny clustered level; retaining the simpler method can be the right decision. A narrative of uninterrupted success teaches little.

Report performance boundaries, memory limits, compatibility gaps, user-test findings and unresolved nonblocking defects. Link each substantial claim to an artifact in the evidence directory. Do not fill the report with fabricated tester quotes or timings. Pending human review stays pending until it occurs.

Oral review and live modification

Be ready to trace one input from device binding through authoritative simulation to render and audio output. Explain a collision normal sign, a rollback restore tick, a callback ownership rule and one rejected mod. Then make a bounded change, such as changing the relay activation threshold, adding a supported key binding or inserting a collision regression test. Predict which tests and documents must change before editing.

An oral review should test understanding, not surprise the learner with unrelated trivia. The capstone specification includes prompts and a scoring rubric. A failed explanation leads to targeted repair and another attempt; attendance and quiz recall alone do not establish mastery.

Completion and the next volume

Pass the mandatory release blockers, meet the rubric threshold, and archive the exact reviewed revision. Gate F closes the undergraduate software-engineering capstone. The next established volume is Computer Engineering and Digital Design, Modules 31-35, not another game-feature expansion. Keep the software workload and measurement discipline: they will provide concrete motivation for later digital-design and hardware/software co-design work.

Original worked checkpoint

A small game with reproducible builds, bounded memory, clear controls and an honest bug list is stronger evidence of engineering ownership than an unfinished large world with no tests.

Full practical assignment

Complete Gate F and the oral questions in the handbook. Pass: another tester finishes a full loop, the release is reproducible, critical defects are resolved or clearly blocked from release, and the postmortem cites measured evidence.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Evidence-gate rubric evaluator**. Run `starter/python/demos/lesson_30_4.py`; implement `starter/python/exercises/lesson_30_4.py`.

Assigned study and native transfer

R20 : course projects and critique; revisit the project rubric and professional-practice lessons.

Revisit R20 project critique and the program Gate F specification. Use the supplied rubric, evidence map and oral review. No external textbook or generated reference implementation substitutes for independent capstone work.

Engine transfer: Archive the reviewed engine/game revision with a clean-build record, an independently completed loop, one reproduced regression and a live-change review. Do not mark human review complete in advance.

Check your understanding

1. What proves completion beyond a recorded demo?
2. Why disclose library and AI assistance?
3. Why include a live modification in an oral review?
4. A game meets its frame budget but an ordinary interrupted save can corrupt the only progress file. Can a high overall rubric score override this?
5. What should a technical postmortem contain beyond a list of completed features?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 30 project

Complete Gate F: the undergraduate software capstone. Integrate one original playable loop with the engine built in Volume 5. This is the culminating use of the module projects, not an additional unrelated large game. Reuse well-understood systems and keep content deliberately small.

Follow CAPSTONE_SPEC for scope, milestone evidence, test matrix and review. The multiplayer demonstration may remain separate from the main game. The game itself still requires collision/physics, animation or articulated motion, bounded audio, inspectable AI, settings, save/load, diagnostics and a tested distribution. Lesson 30.3's data-only mod lab remains required even when the released game does not expose mods.

Collect repeated performance measurements on a declared machine, complete the compatibility/accessibility tasks, test failure recovery and arrange an independent loop completion and live change. A software-renderer route is acceptable where carried forward from Volume 5; do not claim GPU validation without a tested GPU backend.

Deliver the release, source/build instructions, provenance, raw evidence, reviewer record and postmortem. Grade the quality of reasoning and evidence, not art budget, map size or commercial success.

Module 30 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A release's mean frame time improves but its p99 doubles. Is it necessarily better?
2. What is wrong with marking an untested platform as compatible because the code is portable?
3. Does a data-only mod eliminate security and reliability risk?
4. Why keep both a numerical rubric and mandatory release blockers?
5. What belongs in a technical postmortem beyond a feature list?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source